

Sherlock

*Grounded Retrieval-Augmented Generation
over Case Files*



Younus Mashoor

PROJECT REPORT / MAY 27, 2026

PREFACE

This is my report for the Sherlock coding task. Thanks to the Softwerk team for the problem, it was a good one to work through.

AI usage was kept minimal as asked. The only use was generating a few extra synthetic case files to test the system on. I wrote the code by hand and consulted official documentation where needed, listed at the end of the report.

Thanks again for the opportunity.

Younus Mashoor

VÄXJÖ / MAY 27, 2026

1 Overview

Sherlock is a retrieval-augmented question-answering service for case files, exposed as a REST API with a browser frontend. In the task at hand, a detective with thousands of files cannot re-read them all, so Sherlock lets him ask natural-language questions and get answers along with the source document of the answer.

The system does not guess; it abstains. If the case file does not contain the answer, it returns the exact sentence “I don’t have enough evidence to answer that.” instead of fabricating one. This guarantee is enforced through grounding, implemented in two places: the system prompt rules the model follows, and an application-level cross-check on the cited sources. I go into more depth on this in the The Grounding Guarantee section.

For the tech stack, the brief required Python so the backend and supporting layers are all Python; the frontend is the one exception, using plain HTML, CSS, and vanilla JavaScript.

- **Backend:** FastAPI, served by Uvicorn.
- **RAG layer:** LangChain with Chroma as the vector store.
- **Models:** Google Gemini `gemini-embedding-001` and `gemini-2.5-flash-lite`, free tier.
- **Document handling:** `pypdf` and LangChain’s `RecursiveCharacterTextSplitter`.
- **Frontend:** HTML, CSS, vanilla JavaScript.
- **Packaging:** Docker and Docker Compose.
- **Dependencies:** `uv` with a pinned `uv.lock`.
- **Testing:** `pytest`, offline and integration suites.

The remainder of the report covers the architecture, the grounding mechanism, the testing strategy, and the design decisions behind each.

2 Architecture

The system splits into two pipelines that converge on the same vector store. The ingest pipeline runs on file upload: `main.py` saves the file, `system.py` orchestrates the flow, `ingestion.py` chunks the text, and `retrieval.py` embeds the chunks into Chroma.

The query pipeline runs on a question: `main.py` calls `retrieval.py` to fetch the most relevant chunks, passes them to `generation.py` for a grounded Gemini call, then parses the cited sources from the response before returning. The Chroma store is the single point both pipelines touch, and the Gemini API is the only external dependency.

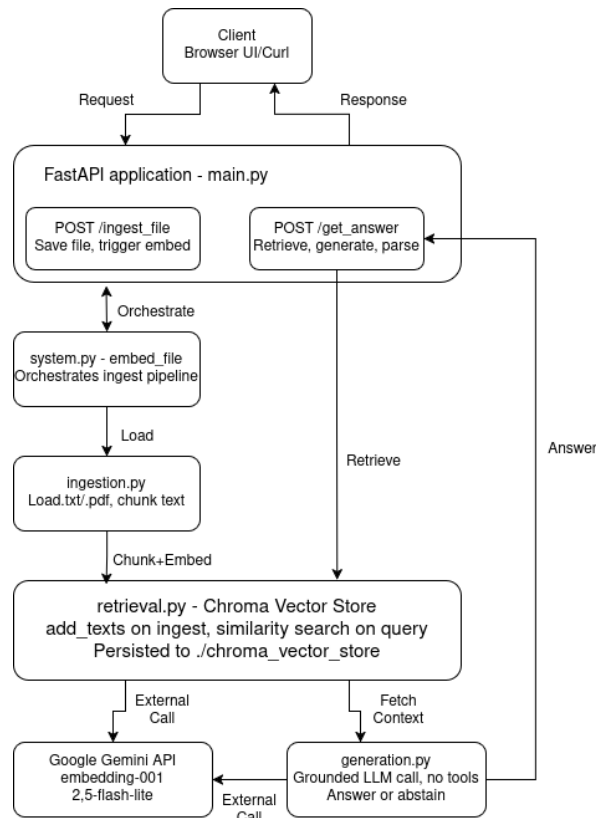


Figure 1. System architecture. The ingest and query pipelines converge on the Chroma vector store.

Every module is pure Python with no HTTP awareness except `main.py`, which talks to the outside world. This makes the rest of the codebase testable in isolation and swappable if FastAPI ever needed to be replaced. My initial thought was Flask, but as the design took shape I realised it was not a necessity and would have added bloat without buying anything, so I dropped it in favour of FastAPI's native async support and Pydantic validation.

The two pipelines are deliberately asymmetric. Ingest goes through `system.py` as an orchestration layer because it is multi-step: load, chunk, hash, then embed. Query bypasses `system.py` and runs directly from `main.py` because it is short enough (one retrieval call followed by one generation call) that an extra layer would have been noise rather than abstraction. I made the call to keep `system.py` narrow rather than forcing both pipelines through it for the sake of symmetry.

A small detail worth flagging: the Gemini clients in both `retrieval.py` and `generation.py` are built lazily via `lru_cache`. This means the modules import cleanly without a Gemini API key set in the environment, which is what allows the offline test suite to import the entire application and patch the external calls out, no network access needed.

Chunks are inserted into Chroma with SHA-256 hashes of their content as their IDs. Re-ingesting the same file therefore produces identical hashes, and Chroma silently overwrites the existing entries rather than duplicating them. It is a small line of code, but it is the reason uploading the same case file twice does not double the size of the result set on every subsequent query.

3 The Grounding Guarantee

The whole project hinges on one rule: Sherlock never makes things up. If the case files don't say it, Sherlock doesn't say it. I enforce that in two layers.

The first layer is the system prompt in `generation.py`. It tells the model exactly when to answer and when to abstain. Figure 2 walks through the decision the model has to make: does the context actually address the question, is the evidence a torn or partial fragment, and is what looks like an answer really just suspicion rather than a direct statement. If any gate fails, the model returns the same fixed sentence every time: "I don't have enough evidence to answer that." I made it a fixed sentence so the frontend can detect it without having to parse arbitrary refusal text.

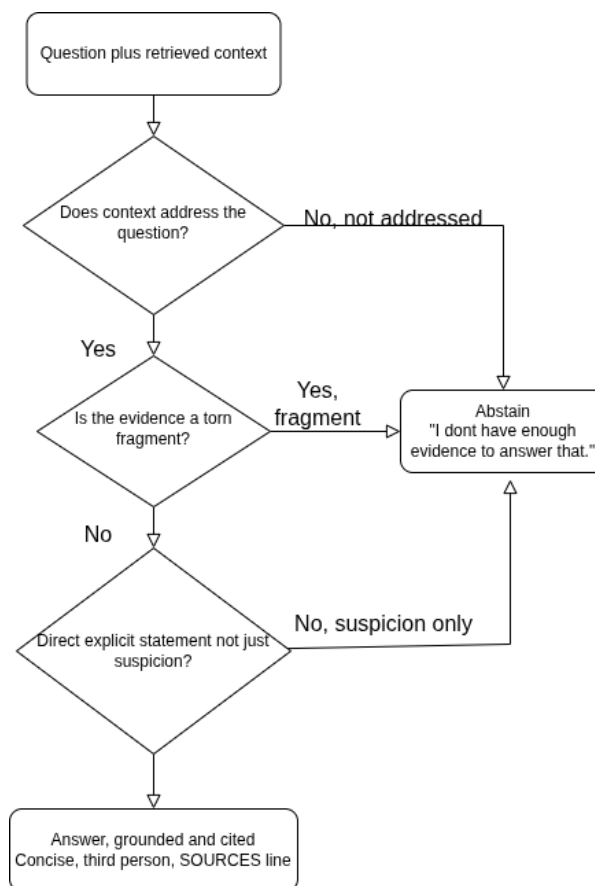


Figure 2. Abstention decision logic. Each gate maps to a numbered rule in the system prompt.

The second layer lives in `main.py`, after the model has replied. Every retrieved chunk gets tagged with its source filename, and the prompt asks the model to end its answer with a **SOURCES:** line listing the files it actually used. `main.py` parses that line and intersects the cited filenames with the list of sources that retrieval actually returned. If the model cites something that wasn't in the retrieved set, it gets dropped before the response goes back to the user.

Both layers have to fail at the same time for a hallucinated answer to escape: the model would have to ignore the prompt rules and invent a citation that happens to match a real filename in the store.

3.1 Known Limitation

The retriever currently returns its top- k chunks for every query regardless of similarity score. There's no relevance floor. If you ask something that has nothing to do with any uploaded file, retrieval still hands the model some chunks, and abstention then rests entirely on the model obeying the prompt rather than on retrieval saying it has nothing useful.

The integration tests confirm the model does hold up on unanswerable questions, so this isn't a bug, but a `score_threshold` retriever would give me a real second line of defence at the retrieval layer. It's first on the future-work list for that reason.

Figure 3 traces the full path of a single `/get_answer` call, from the question hitting the API to the cross-checked sources returning to the client.

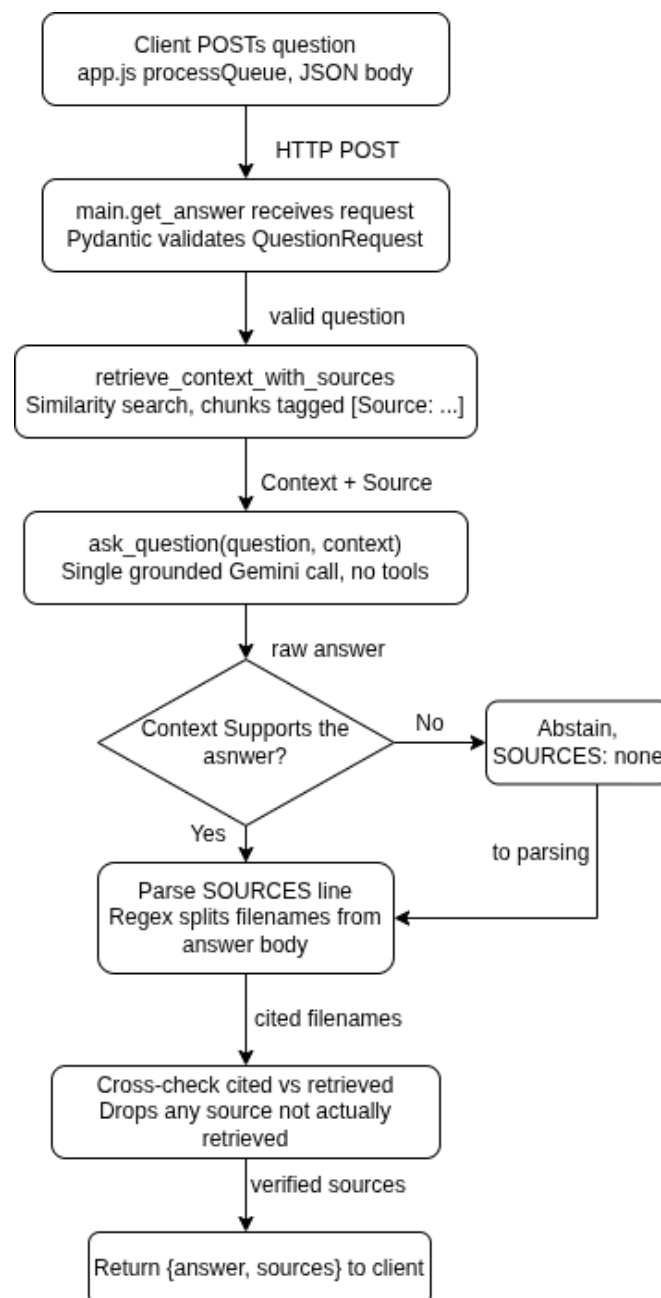


Figure 3. Lifecycle of a `/get_answer` request. The grounding cross-check sits at steps 6 and 7, after the model returns.

4 Implementation Notes

A few decisions in this project deserve a paragraph each, because the choice I made was not always the most popular one available.

Why FastAPI and not Flask

My first instinct was Flask, I've used it before and for a six-endpoint API it would have worked. The thing that tipped it was Pydantic request validation and built-in async, both of which would have meant writing my own models and adding extensions in Flask. For a focused task that felt like the wrong direction.

Why LangChain when there are no agents

I went back and forth on this. The system has no real agent loop, retrieval always runs, then generation always runs, so LangChain is more than I strictly need. What it gives me is a standardised interface across the pieces: the Chroma wrapper, the Gemini embedding client, the text splitter, and the chat model all share the same shape. Swapping any one of them later is a one-line change. I'd rather pay a small abstraction cost now than rewrite half the system if I want to try a different embedding model later.

Chroma with on-disk persistence

Chroma over FAISS because the case files need to survive container restarts. Chroma persists to `./chroma_vector_store` and runs in-process, so there's no second service to manage. It isn't built for multi-instance scaling, but a single-container submission doesn't need that.

SHA-256 chunk IDs for free deduplication

The vector store had to handle the same file being uploaded twice without doubling the result set. Rather than tracking ingested filenames separately, I hash each chunk's content with SHA-256 and use the digest as its ID. Identical chunks produce identical hashes and Chroma silently overwrites instead of inserting duplicates. One line of code, no separate deduplication state.

```
1 ids = [hashlib.sha256(c.encode()).hexdigest() for c in chunks]
2 get_vector_store().add_texts(
3     chunks,
4     metadatas=[{"source": source} * len(chunks),
5     ids=ids)
```

Listing 1. SHA-256 chunk IDs let Chroma deduplicate identical chunks on insert.

Vanilla frontend, no framework

The brief estimates 8 to 12 hours for the whole task and notes that “modern tech stacks are recommended” for the frontend. I read that and decided against it. For a UI with one chat box, one upload form, and one file drawer, React or Svelte would have meant a build step, a bundler, and a node dependency tree to maintain next to the Python backend, all to ship the few hundred lines of behaviour I wrote in vanilla HTML, CSS, and JavaScript. A reviewer skimming for a framework name might mark me down for it. I'd rather defend a small surface area than add tooling because it looks expected.

Client-side question queue

The frontend lets the user fire off questions faster than the model can answer them, so I needed somewhere to hold the in-flight ones. I built a small FIFO queue in `app.js` that buffers questions as they arrive and dispatches them one at a time, rendering each answer back into the chat as it lands. Users can also remove queued messages before they're sent. I went FIFO because the detective will expect answers in the order he asked the questions; LIFO would have felt wrong here.

The queue is also why error handling on the fetch call mattered. Without a try/catch around the dispatch, one failed request would have left `isProcessing` stuck on `true` and the whole queue would have stalled.

Error handling at the boundaries

Every external call is wrapped in error handling, but only at the boundaries where it matters: the four `fetch` calls on the frontend and the two external-dependency calls on the backend (embedding during ingest, generation during query). Internal logic stays unwrapped so a bug surfaces as a stack trace instead of being masked as a vague “API failed”. On the backend, upstream failures return HTTP 502 instead of a generic 500, because the failure is upstream (Gemini), not in our code.

On not overengineering

I'm aware each of the choices above could have gone the other way, and a stricter reviewer might argue some of them should have. The reason they didn't is the same in every case. With 8 to 12 hours budgeted and a strict-grounding guarantee to land, the time I would have spent wiring up a frontend build pipeline or a real agent loop was time better spent on the prompt, the cross-check, and the tests that prove abstention actually works.

An aside on local models

A bit of personal context: I found this project interesting enough that I started building it on my own machine first, using local Ollama models so I could work without a key. I tried `gemma3:latest` for generation alongside `qwen2.5:1.5b` and `qwen2.5:3b`, with `nomic-embed-text` on the retrieval side. The answers themselves were often correct, but the `SOURCES:` discipline broke down: the models would cite two files when only one was actually used, or include a file they hadn't drawn from at all. That's when I appreciated how much of the grounding guarantee depends on the model reliably following the prompt rules, not just generating the right text. The hosted Gemini model held up where the smaller local ones didn't, which is why the submitted version uses it. I'd like to revisit this with a larger local model once I have the hardware for it.

5 API Reference

Method	Path	Purpose
GET	/	Serve the frontend
GET	/health	Liveness check
POST	/ingest_file	Add a case file to the knowledge base
POST	/get_answer	Ask a grounded question
GET	/get_files	List ingested case files
DELETE	/delete_files	Clear all case files and the vector store

Table 1. HTTP endpoints exposed by `main.py`.

6 Testing

I split the tests into two suites: a fast offline one that runs without an API key, and an opt-in integration one that drives the real Gemini API. The grounding guarantee can really only be proven against the live model, but I didn't want the rest of the codebase locked behind a key for CI or quick local checks.

Offline suite

The offline tests cover everything that doesn't need a model call. `test_ingestion.py` checks that the chunker produces non-empty chunks of bounded size and that the document loader raises the right exceptions for missing files and unsupported extensions. The more interesting file is `test_offline.py`, which patches out `get_vector_store`, `retrieve_context_with_sources`, and `ask_question` so the `/get_answer` endpoint can be exercised with fixed inputs. That's what lets me verify the source cross-check actually drops citations the model invents, and that the abstention path returns an empty sources list, without spending any quota.

Integration suite

The integration suite is in `test_grounding.py`, with a few integration-marked endpoint tests in `test_api.py`. `test_grounding.py` ingests the three case files into an isolated in-memory Chroma collection and runs two parameterised batteries. The first checks that answerable questions produce answers containing the expected keywords. The second checks that questions whose answers are not in the case files return the exact abstention sentence. The cases cover all three documents, including a deliberate cross-document one (the Albert Finch question pulls from a different file than the others) to confirm retrieval actually spans sources rather than just hitting whichever document is most recent.

Test data

The case files in `case_files/` were generated with AI assistance, the one declared use of AI in the project. I generated them rather than wrote them by hand because I needed documents with specific properties to test against: known answerable questions, deliberate unanswerable ones, a torn-fragment scenario for one of the prompt rules, and a clear suspect-versus-evidence distinction for another. A handwritten case file would have taken longer and probably covered fewer of the edge cases I wanted to hit.

For larger inputs I also dropped four public-domain Sherlock Holmes short stories (two as `.txt`, two as `.pdf`, in `tests/novels/`) into the test directory. They gave me longer, real-world documents to confirm chunking and retrieval behave sensibly at the kind of length a real case archive would involve, rather than the few hundred lines of the synthetic files.

7 Running the Project

The README also has the setup, but in short run the following two commands. Add a Gemini API key to a `.env` file and bring the container up:

```
1 cp .env_example .env # add your GEMINI_API_KEY
2 docker compose up --build # serves on http://localhost:8080
```

Listing 2. Build and run via Docker.

The frontend is served from the same container at the root path, so `http://localhost:8080` is the only URL you need.

8 Self-Assessment Against the Criteria

Criterion	Where it is addressed
Solved	End-to-end ingest plus grounded Q&A running in Docker; integration tests cover both answer and abstention paths against seven answerable and four unanswerable questions.
Code	Clear module boundaries, type hints on public functions, no debug leftovers. Lazy client initialisation so modules import without an API key.
Architecture	Two-pipeline design with the vector store as the single convergence point; HTTP boundary isolated to <code>main.py</code> ; deliberate orchestration asymmetry between ingest and query.
Modern technologies	FastAPI, LangChain, Chroma, Gemini, uv for dependencies, Docker Compose for orchestration.
Framework leverage	LangChain's standardised interfaces across vector store, embeddings, splitter and chat model; FastAPI's native Pydantic validation and async.
Documentation	This report plus the README, both with three diagrams; explanatory comments on the non-obvious functions and docstrings across the test suite.
Test	Two suites (offline, integration). <code>test_grounding.py</code> parameterises eleven cases to verify the core guarantee; <code>test_offline.py</code> mocks Gemini to test the source cross-check deterministically.
Bonus	A second grounding layer via <code>SOURCES</code> : cross-check; client-side FIFO question queue with per-message cancellation; error handling at every external boundary; three architecture diagrams; local-model exploration documented in Implementation Notes.

Table 2. How the submission maps to the scoring aspects.

9 Future Work

A longer list than the usual two or three bullets, because I want to be clear about what I'm aware of versus what I shipped. Most of these are deliberate scope decisions for a 12-hour task rather than gaps I missed. Listed roughly in priority order.

- **Retrieval-side similarity threshold.** The retriever currently hands the model its top- k chunks regardless of how relevant they actually are. Adding a `score_threshold` would let retrieval itself say “I have nothing useful here”, instead of leaving the model as the only

thing standing between an irrelevant question and a wrong answer.

- **Filename sanitisation on upload.** The `/ingest_file` endpoint writes the user-supplied filename straight to disk, so a path-traversal name like `../etc/passwd` would work as far as the file write goes. `Path(file.filename).name` closes it in one line. A real security gap I saw and didn't fix in scope.
- **Real agentic RAG, not a chain.** I deliberately built this as a deterministic chain rather than an agent loop, because for strict grounding determinism is a feature. But there's a real version where the model decides *when* to retrieve and *what* to retrieve, possibly across multiple passes, which would handle multi-hop questions ("what alibi did the person who held the key give?") better than a single retrieval ever can. That's the version that earns LangChain's agent machinery.
- **Reranking the retrieved chunks.** Cosine similarity over embeddings is the cheapest retrieval signal, not the best one. A cross-encoder reranker over the top-20 retrieved chunks would catch cases where the right chunk is semantically nearby but not the literal nearest neighbour. It's a known weakness of vanilla RAG and the standard mitigation.
- **Hybrid retrieval, dense plus sparse.** Pure vector search loses on exact-match queries (specific names, dates, quoted phrases). Pairing the dense retriever with BM25 and fusing the results catches both "what was Mrs. Hudson's alibi" (semantic) and "mention of the south gate" (lexical) reliably. Most production RAG systems do this; mine does not.
- **Streaming answers to the frontend.** The UI currently blocks on the full response. Streaming the model output as it arrives would make the wait feel faster and is a short change with FastAPI's `StreamingResponse`.
- **Persisting the upload pipeline outside the container.** Chroma writes to a volume, but the case files themselves live on the container filesystem alongside the app. A production version would push uploads to object storage (S3-compatible) so the API can scale horizontally without sharing local disk. This is a container-architecture limitation, not a code one.
- **Observability.** There are no structured logs, no request tracing, and no metrics on retrieval quality or abstention rate. For a task of this size that's reasonable, but any real deployment would want at minimum a count of abstentions over time, since a sudden spike in abstentions usually means ingestion or retrieval has regressed.
- **Better use of the framework I picked.** I use LangChain for its interfaces but not for its expression language (LCEL), which would let the retrieve-then-generate pipeline be declared as a single composed runnable rather than two function calls in `main.py`. If the system grows further (multi-step retrieval, conditional routing, fallback chains), that's the refactor that pays off.
- **Frontend, eventually.** Vanilla HTML and JavaScript was the right call for the scope here, but a Svelte or React frontend would make features like streaming responses, source highlighting inside the original document, and a proper file viewer much easier to build than they would be in plain JavaScript.

The thread through this list is the same one running through Implementation Notes: I picked the smaller option whenever the bigger one wasn't earning its keep for the task as scoped. Everything above is a place where that calculation would change if the project grew beyond a single-container submission.

References

- FastAPI documentation. <https://fastapi.tiangolo.com/>
- LangChain documentation. <https://docs.langchain.com/>
- Chroma documentation. <https://docs.trychroma.com/docs/overview/getting-started>
- pypdf documentation. <https://pypdf.readthedocs.io/>
- Google AI Studio (Gemini API). <https://aistudio.google.com/>
- Ollama (local-model exploration). <https://ollama.com/>
- Docker Compose reference. <https://docs.docker.com/compose/>
- Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>
- uv (Astral) documentation. <https://docs.astral.sh/uv/>
- pytest documentation. <https://docs.pytest.org/>
- Python os module. <https://docs.python.org/3/library/os.html>
- Python shutil module. <https://docs.python.org/3/library/shutil.html>
- W3Schools sliding navbar example. https://www.w3schools.com/howto/howto_js_navbar_slide.asp